

Chapter 2. Q-learning

Samsung AI Expert
Reinforcement Learning

Jul 8, 2025

Tabular Q-learning

MDP definition

```
1  class MyEnv:
2      num_actions = 4
3
4      def __init__(self):
5          pass
6
7      def reset(self):
8          pass
9
10     def step(self, action):
11         pass
```

s_t is kept internally, and is updated in **step** method

sample an initial state $s_0 \sim \rho_0(s)$

agent-env interaction:

$s_{t+1} \sim p(\cdot | s_t, a_t), r_t = r(s_t, a_t)$

Tabular Q-learning

Example - Pendulum

```
1  class PendulumEnv(gym.Env):
2      metadata = {
3          'render.modes': ['human', 'rgb_array'],
4          'video.frames_per_second': 30
5      }
6
7      def __init__(self, g=10.0):
8          self.max_speed = 8
9          self.max_torque = 2.
10         self.dt = .05
11         self.g = g
12         self.m = 1.
13         self.l = 1.
14         self.viewer = None
15
16         high = np.array([1., 1., self.max_speed], dtype=np.float32)
17         self.action_space = spaces.Box(
18             low=-self.max_torque,
19             high=self.max_torque, shape=(1,),
20             dtype=np.float32
21         )
```

MDP as a Python class(gym.Env)

https://github.com/openai/gym/blob/master/gym/envs/classic_control/pendulum.py



Tabular Q-learning

Example - Pendulum

action input a_t (torque applied at t)

```
1 def step(self, u):  
2     th, thdot = self.state # th := theta  
3  
4     g = self.g  
5     m = self.m  
6     l = self.l  
7     dt = self.dt  
8  
9     u = np.clip(u, -self.max_torque, self.max_torque)[0]  
10    self.last_u = u # for rendering  
11    costs = angle_normalize(th) ** 2 + .1 * thdot ** 2 + .001 * (u ** 2)  
12  
13    newthdot = thdot + (-3 * g / (2 * l) * np.sin(th + np.pi) + 3. / (m * l ** 2) * u) * dt  
14    newth = th + newthdot * dt  
15    newthdot = np.clip(newthdot, -self.max_speed, self.max_speed)  
16  
17    self.state = np.array([newth, newthdot])  
18    return self._get_obs(), -costs, False, {}
```

state s_t : $s_t = (\theta_t, \dot{\theta}_t)$

compute reward $r_t = r(s_t, a_t)$

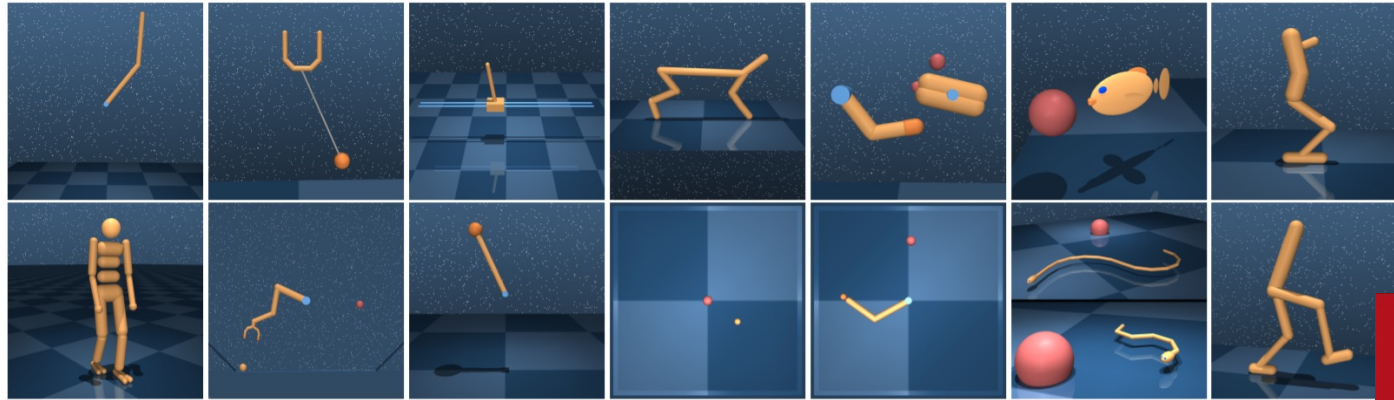
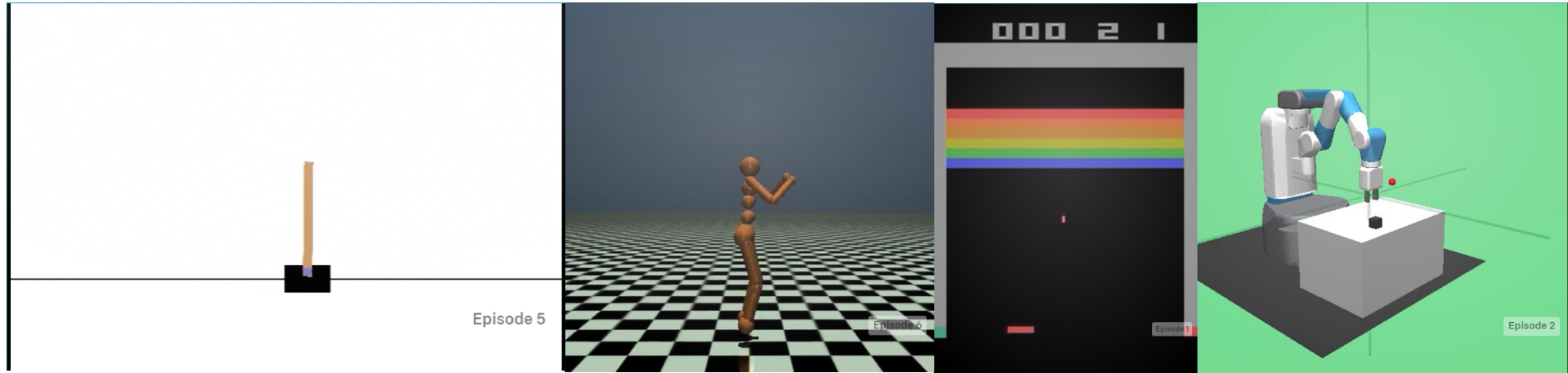
compute the next state s_{t+1}

return next state s_{t+1} & reward

https://github.com/openai/gym/blob/master/gym/envs/classic_control/pendulum.py

Tabular Q-learning

More examples - OpenAI Gym, Deepmind Control Suite, etc.



Learning and Decision Systems

TODO_1)

TODO_2)



Q-learning - Implementation

Algorithm Implementation

```
1  class QTable:
2      def __init__(self, num_states, num_actions, gamma=0.99):
3          self.gamma = gamma
4          self.Q = np.zeros(shape=(num_states, num_actions))
5
6      def update(self, state, action, reward, next_state, alpha):
7          target = reward + self.gamma * np.max(self.Q[next_state]) - self.Q[state, action]
8          self.Q[state, action] += alpha * target
9
10     def act(self, state):
11         return np.argmax(self.Q[state])
```

store Q-function as a table

Q-learning update!

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t(s_t, a_t) \underbrace{\left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)}_{\text{error}}$$

greedy action $a_t = \arg \max_a Q(s_t, a)$

TODO_1)

TODO_2)



TODO_1)

```
target = reward + self.gamma * np.max(self.Q[next_state]) - self.Q[state, action]
```

TODO_2)

```
self.Q[state, action] += alpha * target
```

TODO_3)



```
TODO_3)
```

```
return np.argmax(self.Q[state])
```

Q-learning - Implementation

Algorithm Implementation

```
1  class QTable:
2      def __init__(self, num_states, num_actions, gamma=0.99):
3          self.gamma = gamma
4          self.Q = np.zeros(shape=(num_states, num_actions))
5
6      def update(self, state, action, reward, next_state, alpha):
7          target = reward + self.gamma * np.max(self.Q[next_state]) - self.Q[state, action]
8          self.Q[state, action] += alpha * target
9
10     def act(self, state):
11         return np.argmax(self.Q[state])
```

store Q-function as a table

Q-learning update!

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t(s_t, a_t) \underbrace{\left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)}_{\text{error}}$$

greedy action $a_t = \arg \max_a Q(s_t, a)$

TODO_4)



Learning and Decision Systems

TODO_4)

```
if u < epsilon_schedule(t):  
    a = env.action_space.sample()  
else:  
    a = learner.act(state=s)
```

TODO_5)

TODO_6)



Learning and Decision Systems

TODO_5)

```
s_next, r, _, _ = env.step(action=a)
```

TODO_6)

```
learner.update(state=s, action=a, reward=r, next_state=s_next, alpha=alpha_schedule(n))
```


Q-learning - Implementation

Complete Outline

```
1 learner = QTable(num_states=env.observation_space.n, num_actions=env.action_space.n, gamma=gamma)
2 rollout_len = 1000000
3 visit_count = np.zeros(shape=(num_states, num_actions)) # save visit counts N(s, a) of all state-action pairs
4 alpha = VisitCountStepsizeSchedule(deg=0.5001)
5 epsilon = LinearExplorationSchedule(rollout_len, final_epsilon=0.4)
6
7 s = env.reset()
8 for t in tqdm(range(rollout_len + 1)):
9     u = np.random.rand()
10    if u < epsilon(t):
11        a = env.action_space.sample()
12    else:
13        a = learner.act(state=s)
14    s_next, r, _, _ = env.step(action=a)
15    n = visit_count[s, a]
16    learner.update(state=s, action=a, reward=r, next_state=s_next, alpha=alpha(n))
17    visit_count[s, a] += 1
18    s = s_next
```

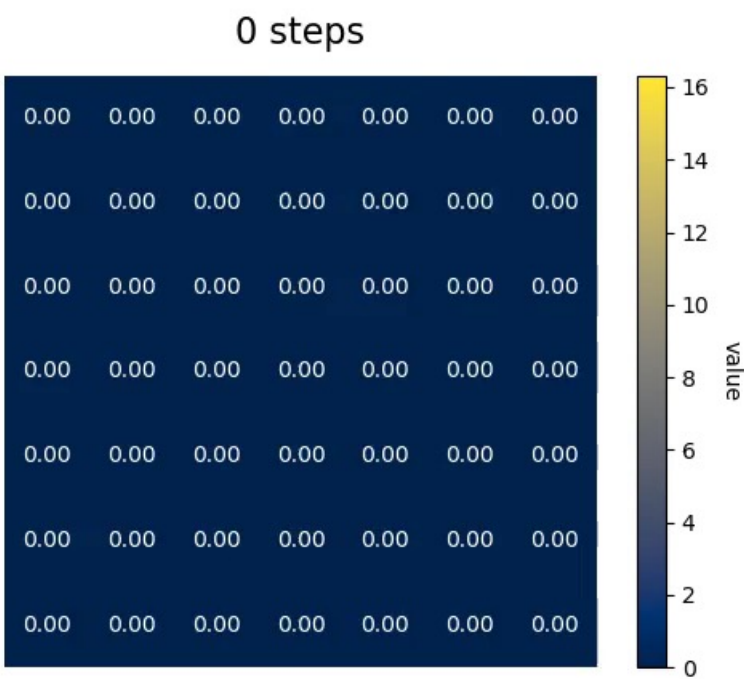
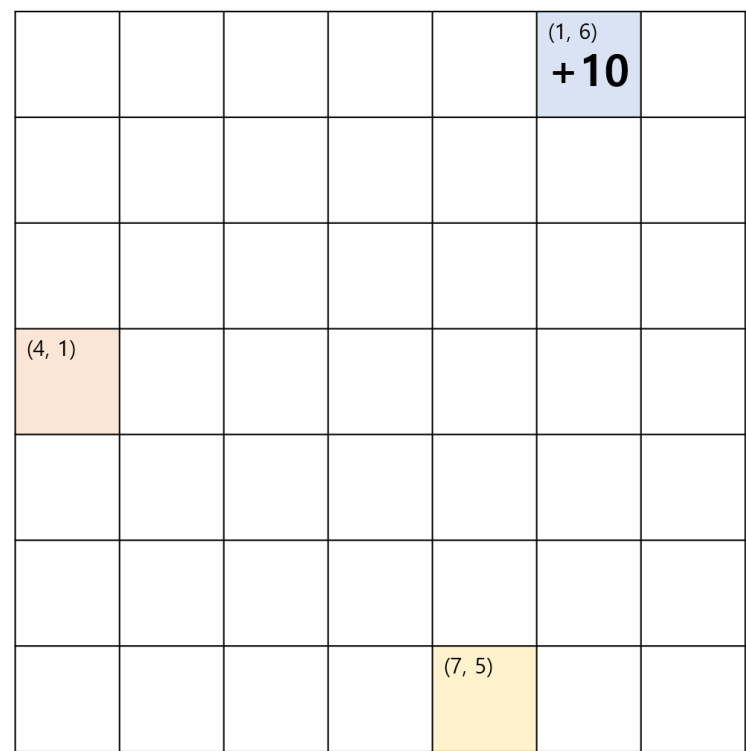
stepsize rule : $\alpha_t(s, a) = \frac{1}{n_t(s, a)^d}$
 $1/2 < d < 1$

exploration strategy : start with large ε , and decrease it.

ε -greedy action selection

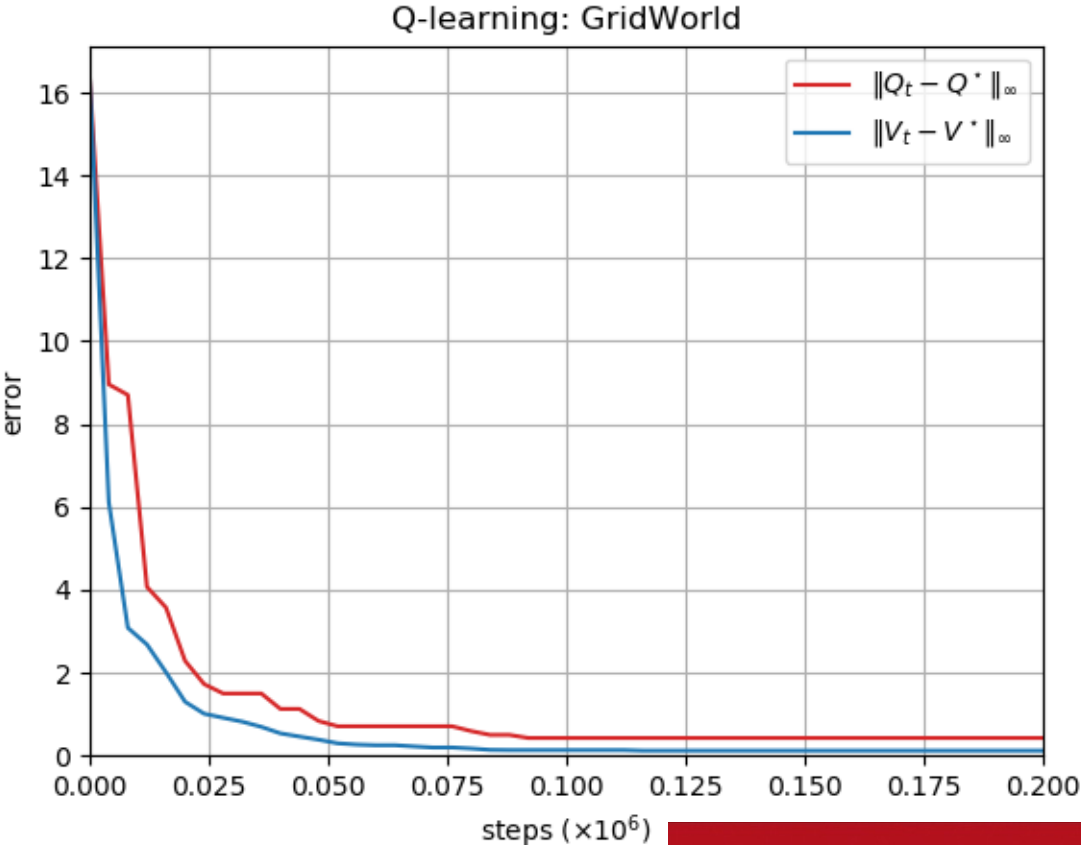
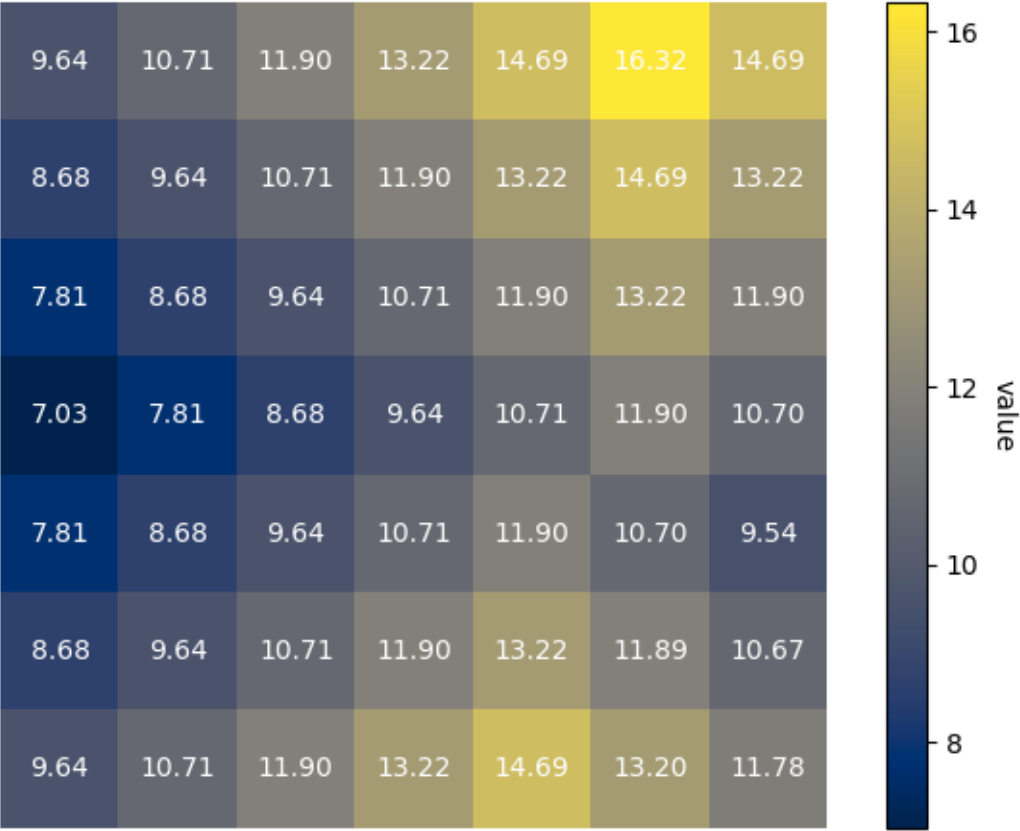
Practice1 - GridWorld

In Day 1, we learned how to compute the value function of GridWorld via **value iteration** when the model is completely **known**.



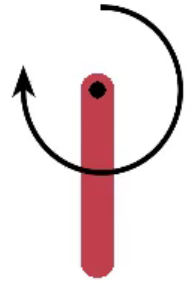
Practice1 - GridWorld

Q-learning: GridWorld



Practice2 – Discrete Pendulum

- discretize pendulum dynamics
 - 2-dim. state $(\theta, \dot{\theta})$
 - 1-dim action τ
- goal : apply a torque τ to a joint for swing-up
- 2460 discretized states & 15 discretized actions



TODO_7)



Learning and Decision Systems

```
TODO_7)
```

```
env = gym.make('DiscretePendulum-v1')
```

TODO_8)

TODO_9)



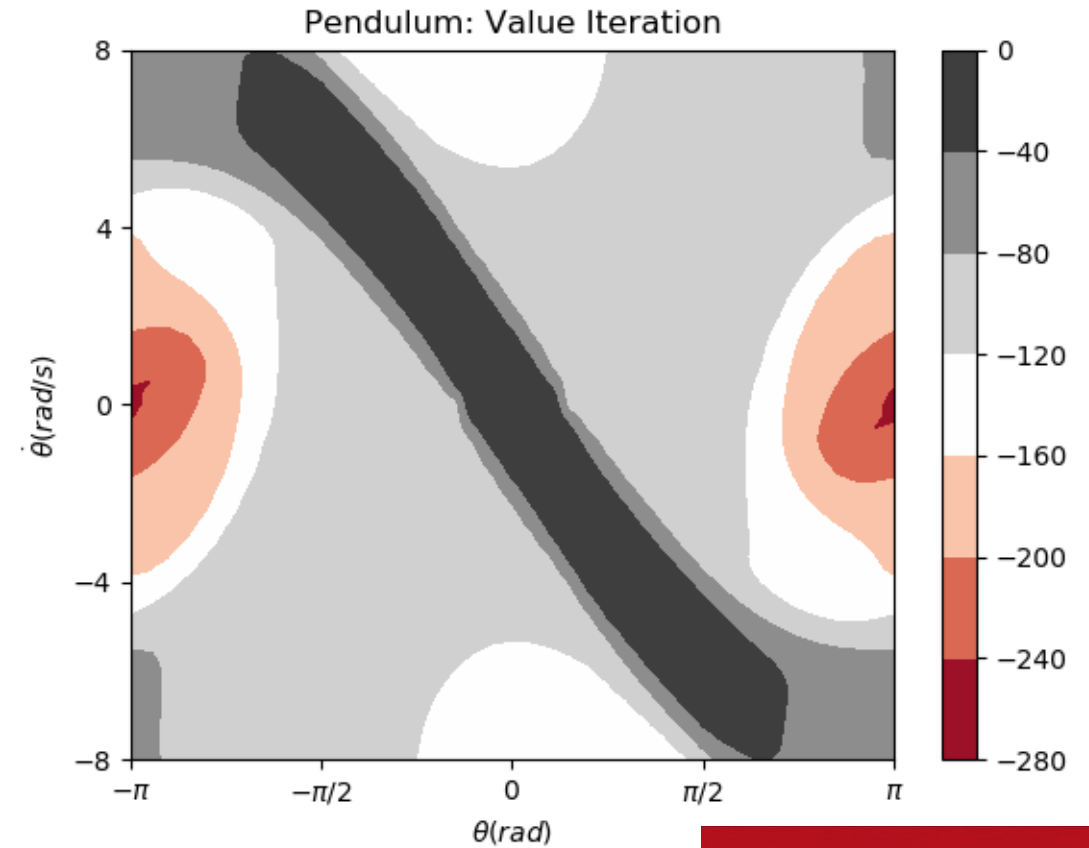
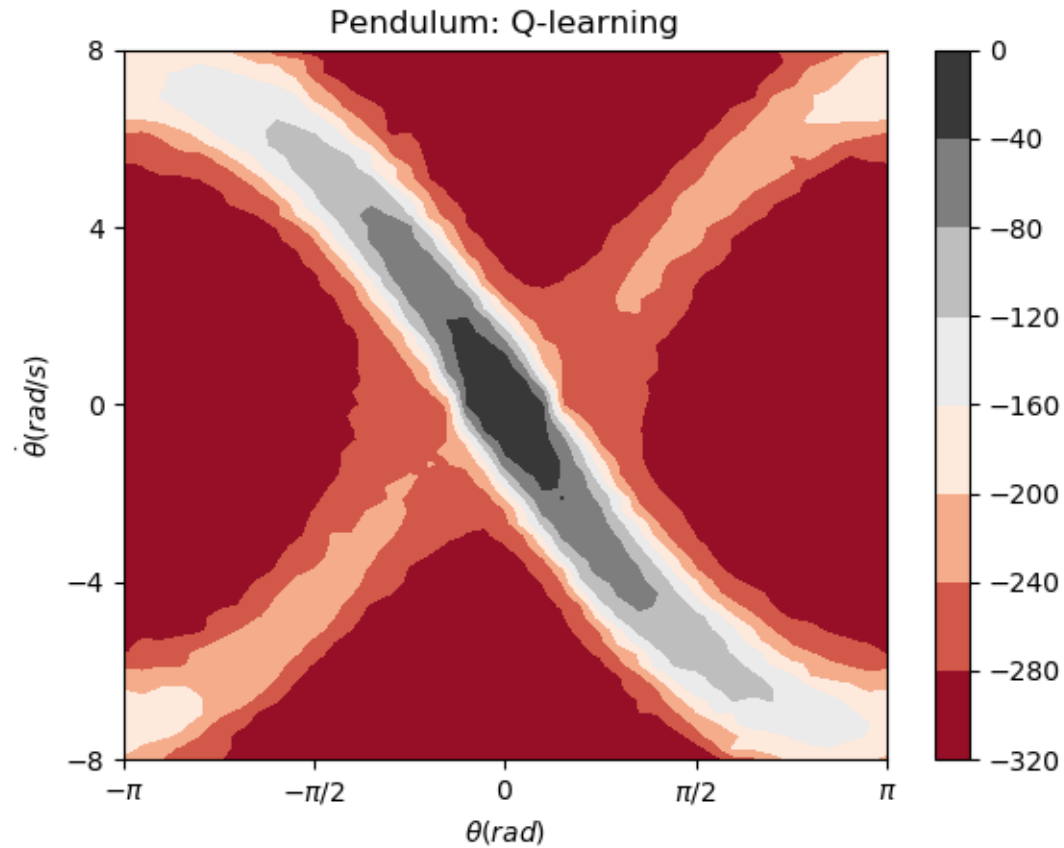
TODO_8)

```
a = learner.act(s)
```

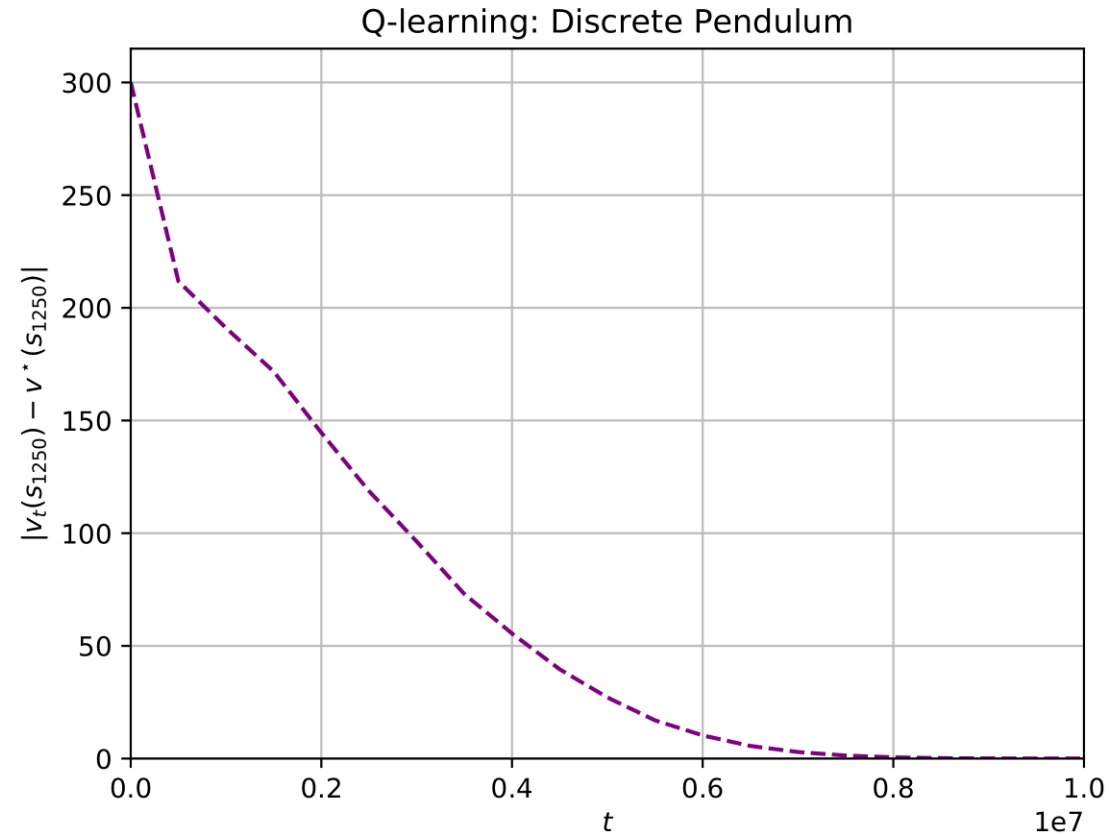
TODO_9)

```
reward += (gamma ** t) * r
```


Practice2 – Discrete Pendulum



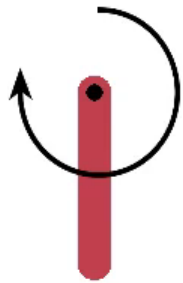
Practice2 – Discrete Pendulum



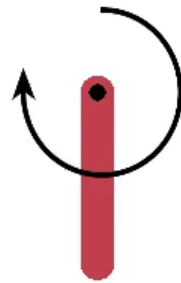
Why Deep Q-Network?

- It seems like Q-learning works well in these examples.
- Why **deep reinforcement learning** then?

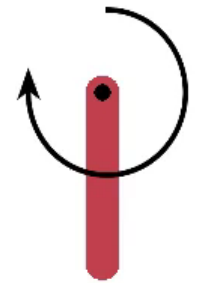
⇒ For most real-world problems, discretization is not a good strategy...



$n = 160, m = 5$ (**coarse**)



$n = 620, m = 10$



$n = 2460, m = 15$ (**fine**)

LDS

Learning and Decision Systems

Thank you!



Learning and Decision Systems